

workable-items — Operator Guide

Revision: 1 **Last modified:** 2026-05-28T17:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator-facing guide for the v1.0.15 project-local workable-items Go binary — the §11.4.93 SQLite Single-Source-of-Truth for every workable item tracked in `Issues.md` / `Fixed.md`.

1. What this is

`cmd/workable-items/` is a small Go binary that owns the canonical form of every workable item in the tmux project. The on-disk truth is `docs/workable_items.db` — a SQLite file **tracked in git per §11.4.95**. Markdown (`Issues.md` / `Fixed.md`) is regenerated FROM the DB; the DB is regenerated FROM the Markdown. Round-trip drift is detected by `workable-items diff`.

Two design constraints govern the layout:

- **§11.4.93 SQLite-SSoT mandate.** Workable items must live in a structured, queryable store — not free-form Markdown — so that validation (§11.4.33 closure vocabulary, §11.4.91 description clarity, §11.4.5 / §11.4.69 evidence-required-on-close) can be enforced mechanically.
- **§11.4.95 DB tracked in git mandate.** `docs/workable_items.db` is checked into git on every commit. Operators inspecting the tracker get a deterministic snapshot per SHA; there is no "DB on the server, Markdown in git" split-brain.

The constitution submodule (`constitution/scripts/workable-items/`) ships a Phase-2 scaffold (stubs + canonical `schema.sql`). The Phase 3+ implementation lives in THIS repo at `cmd/workable-items/` per the `feedback_no_modify_constitution` memory rule. Upstream PR to `HelixDevelopment/HelixConstitution` is tracked as a follow-up.

2. Quick start

```
# Build (no CGO, no system SQLite required)
go build ./cmd/workable-items
# → produces ./workable-items in CWD

# Show every item in the DB
./cmd/workable-items/workable-items report

# Validate the DB against §11.4 mandates
./cmd/workable-items/workable-items validate
```

```

# Detect Markdown ↔ DB drift (exits 1 on drift)
./cmd/workable-items/workable-items diff

# Sync DB → regenerated Markdown (writes under docs/workable-
  items/regen/)
./cmd/workable-items/workable-items sync db-to-md

# Sync Markdown → DB (re-parse Issues.md + Fixed.md, allocate ATM-
  NNNs)
./cmd/workable-items/workable-items sync md-to-db

# Add a new item
./cmd/workable-items/workable-items add \
  --type Bug --severity HIGH \
  --title "Wheel scroll broken on Termux" \
  --description "Pinch-to-zoom intercepts touch before tmux sees
    wheel events; need explicit binding for Termux's gesture
    handler" \
  --category A

# Close an item with evidence (§11.4.5 + §11.4.69)
./cmd/workable-items/workable-items close ATM-042 \
  --status fixed \
  --evidence qa-results/runs/2026-05-28T17-00-00Z/test_44.log \
  --by AI --on 2026-05-28

```

Run `workable-items --help` for the full subcommand reference.

3. Where things live

Path	Role
cmd/workable-items/	Go source (11 files: model / parser / db / sync_md_to_db / sync_db_to_md / diff / validate / add / close / main + tests)
cmd/workable-items/ schema.sql	Embedded schema — drift-checked verbatim copy of constitution/scripts/workable-items/schema.sql@6828ff2
cmd/workable-items/ testdata/	Golden corpus (golden_issues.md / golden_fixed.md + exports) for round-trip byte-identical tests
docs/workable_items.db	The canonical DB — TRACKED in git per §11.4.95
docs/workable-items/ Status.md	Project-local Status document (§11.4.45)
docs/workable-items/ Status_Summary.md	Two-audience companion (§11.4.56)
docs/workable-items/ regen/	Output of sync db-to-md (gitignored — regenerate on demand)

4. Subcommands

Subcommand	Purpose
sync md-to-db	Parse <code>Issues.md</code> + <code>Fixed.md</code> , upsert rows in the DB, allocate ATM-NNN IDs for new items. Idempotent: a second invocation with no Markdown changes allocates 0 new IDs.
sync db-to-md	Render the DB back to Markdown under <code>--out-dir</code> . Byte-identical to source for the golden testdata corpus; lossy for live free-form bodies (see §6).
diff	Open a temp DB, sync <code>Issues.md</code> + <code>Fixed.md</code> into it, compare against the live DB. Exits 1 on drift unless <code>--allow-drift</code> .
validate	Walk the DB and report violations: §11.4.33 type-aware closure vocabulary, §11.4.91 description-clarity floor (≥ 40 chars or ≥ 6 words), §11.4.5 + §11.4.69 evidence-required for closed items. Exits 1 if any findings.
add	Append a new item. ATM-NNN allocated monotonically. Required: <code>--type</code> , <code>--severity</code> , <code>--title</code> , <code>--description</code> .
close	Mark an item closed. Required: <code>--status</code> <code>fixed implemented completed obsolete</code> , <code>--evidence</code> <code>PATH</code> . Per §11.4.90, <code>obsolete</code> additionally requires <code>--superseding-item</code> and <code>--triple-check-evidence</code> .
report	List items, optionally filtered by <code>--type</code> , <code>--status</code> , or <code>--obsolete-audit</code> .

5. Why a Go binary (not a shell script)

- **No CGO.** Uses modernnc.org/sqlite — a pure-Go port of SQLite 3.53.1 (v1.51.0 as of 2026-05-28). Cross-compiles on Mistborn (Darwin arm64) and nezha (Linux x86_64) with no build-host SQLite dependency.
- **Atomic writes via WAL.** OpenDB enables WAL mode; Close runs `PRAGMA wal_checkpoint(TRUNCATE)` so the on-disk file is always self-contained. Compatible with `git add` (no `-wal` / `-shm` files left behind for git to choke on).
- **Embedded schema.** `cmd/workable-items/schema.sql` is `//go:embed-ed` into the binary. The DB applies the embedded schema on OpenDB if not already applied. A drift-check header on line 1 records the upstream constitution SHA the schema was copied from.
- **go test ./cmd/workable-items/... -count=3** runs deterministically per §11.4.50 — 10 tests $\times$ 3 iterations = 30 PASS.

6. Honest gaps (§11.4.6 — no guessing)

The following are KNOWN gaps, tracked transparently. None of them is a §11.4 PASS-bluff: each is a documented topology / scope limitation with a follow-up plan.

1. **Live-corpus round-trip is NOT byte-identical.** `Issues.md` / `Fixed.md` carry free-form bodies (forensic anchors, multi-paragraph "Source-side fix" + "Captured evidence" sections, blockquotes, code fences) that cannot be losslessly reconstructed from the structured items schema's flat description field. The §11.4.93 phase-6 migration plan addresses this; until then, round-trip byte-identical equivalence holds for the **golden testdata corpus only**, and `sync db-to-md` against live data produces simplified bodies under `docs/workable-items/regen/` that are intentionally not used to overwrite the canonical `Issues.md` / `Fixed.md`.
2. **45 legacy items default to Type=Task.** The `tmux` corpus predates §11.4.16 — no `**Type:**` lines exist in the original `Issues` / `Fixed` bodies. `workable-items validate` therefore reports §11.4.33 violations for every closed item that used the `RESOLVED` heading hint (mapped to `Fixed` (→ `Fixed.md`), the Bug closure word) while the row's type is `Task` (which would expect `Completed` (→ `Fixed.md`)). This is the **expected initial state** of the post-population DB, not a bug in the binary. `PWU-Q5` (legacy-items `Type=Bug` cleanup) is the closure path.
3. **Phase 3+ implementation is project-local.** Per the `feedback_no_modify_constitution` memory rule (no project-local pushes into the constitution submodule), this implementation lives at `cmd/workable-items/`. Upstream PR to `HelixDevelopment/HelixConstitution` is operator-blocked pending explicit authorisation. The constitution's Phase-2 scaffold and the project's Phase-3+ implementation share the schema verbatim (drift-checked on `OpenDB`).
4. **`commit_all.sh` + `sync_issues_docs.sh` integration is owned by `PWU-Q4`** (parallel cycle, 2026-05-28). Until that `PWU` lands, operators must invoke `workable-items sync md-to-db` manually after editing `Issues.md` / `Fixed.md` and re-commit the updated `docs/workable_items.db`.

7. Anti-bluff verification

Layer	Mechanism	Where
1 — pre-build gate	<code>verify.sh</code> greps for <code>cmd/workable-items/main.go</code> + <code>docs/workable_items.db</code> present	<code>scripts/verify.sh</code>
2 — runtime	<code>go test ./cmd/workable-items/... -count=3</code> (30 PASS / 0 FAIL)	<code>cmd/workable-items/*_test.go</code> <code>scripts/challenges/tmux.yaml</code>

Layer	Mechanism	Where
3 — Challenge	CME-WORKABLE-ITEMS-001 (planned, PWU-Q6)	
4 — paired mutation	M-WORKABLE-ITEMS (planned, PWU-Q6) — strips the §11.4.91 clarity floor and asserts validate no longer FAILs on a 1-word description	scripts/tests/ meta_test_false_positive_proof.sh

8. Cross-references

- [Constitution.md](#) §11.4.93 (SQLite-SSoT), §11.4.95 (DB TRACKED in git), §11.4.33 (type-aware closure), §11.4.91 (description clarity), §11.4.50 (deterministic `-count=3`), §11.4.69 (sink-side evidence applied to `--evidence`)
- [docs/workable-items/Status.md](#) — live status per §11.4.45
- [docs/workable-items/Status_Summary.md](#) — two-audience companion per §11.4.56
- [CHANGELOG.md](#) v1.0.15 — A39 release notes
- Constitution scaffold (Phase 2): `constitution/scripts/workable-items/`

Sources verified 2026-05-28

- **Constitution §11.4.93 / §11.4.95 / §11.4.50 / §11.4.33 / §11.4.69** — `constitution/Constitution.md` @ submodule pointer 6828ff2 (pulled 2026-05-28 per §11.4.37 `fetch-before-edit`).
- **modernc.org/sqlite** — <https://pkg.go.dev/modernc.org/sqlite> (verified 2026-05-28). Latest published version v1.51.0 (2026-05-28), ships SQLite 3.53.1. Confirmed: pure-Go (CGo-free) driver, import path `modernc.org/sqlite`, driver name `sqlite` for `sql.Open`, WAL + PRAGMA `wal_checkpoint` supported via `_pragma` query param or via direct PRAGMA statements.
- **Constitution Phase-2 scaffold** — `constitution/scripts/workable-items/schema.sql` @ 6828ff2; verbatim drift-checked copy at `cmd/workable-items/schema.sql` header line 1.
- **§11.4.99 sources-verification mandate** — new clause landed in `constitution` 9e3bcc5 (2026-05-28), short-form mirror in this project's `Constitution.md` / `CLAUDE.md` / `AGENTS.md` / `QWEN.md` (PWU-Q5 owns the propagation row).